

Projeto SmartFactory Lab

Versão AWS

Objetivo

Demonstrar de forma prática:

- Pods
- Deployments
- Services
- ConfigMaps
- Secrets
- Volumes Persistentes
- Ingress
- Escalabilidade horizontal
- Balanceamento de carga
- Falha e auto-recuperação
- Logs
- Monitoramento
- Comunicação entre containers
- Arquitetura distribuída
- Containers
- Alta disponibilidade
- Banco de dados
- Autenticação
- Observabilidade
- Mensageria
- Concorrência distribuída

- Rolling update

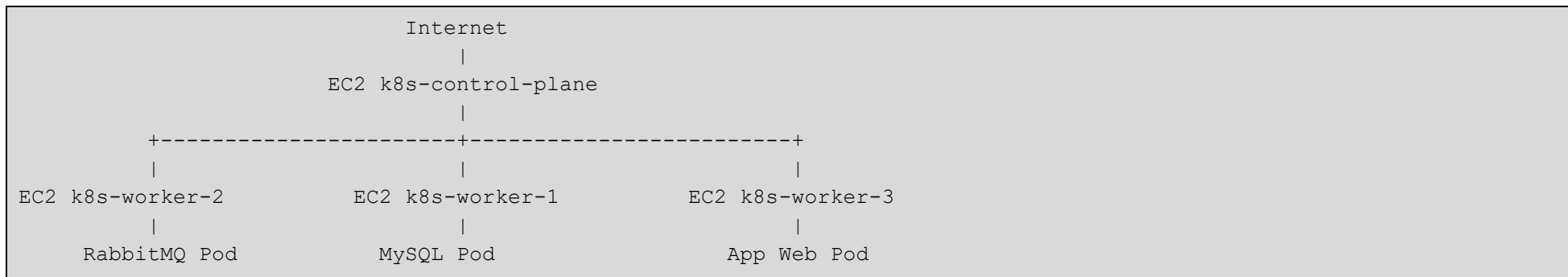
Cenário

Uma empresa com:

- Website institucional
- Login centralizado
- Banco de dados
- Servidor de arquivos
- Dashboard de métricas
- Fila de mensagens
- Cache
- Backup automático

Arquitetura

TODO: fazer o desenho da Arquitetura Visual



Árvore de Arquivos

```
SmartFactory_Lab/
├── k8s
│   ├── ingress.yaml
│   ├── mysql.yaml
│   ├── namespace.yaml
│   ├── rabbitmq.yaml
│   └── web.yaml
├── README.md
├── web
│   ├── Dockerfile
│   ├── factoryflow
│   │   ├── settings.py
│   │   ├── urls.py
│   │   └── views.py
│   ├── manage.py
│   └── requirements.txt
```

Primeira Parte: infraestrutura na AWS

Vamos usar **EC2** da **AWS**. Nesse projeto, não vamos usar **EKS**.

EC2

Criar uma **EC2**:

- **Ubuntu Server LTS (26.04 LTS) - Free tier eligible**
- **8 vCPU e 8 GB** de memória (**t2.large**) - ou equivalente mais barato

Abrir a console **EC2** e clicar em **Launch instance**.

Launch instance

To get started, launch an Amazon EC2 instance, which is a virtual server in the cloud.

Launch instance ▼

Migrate a server ↗

Note: Your instances will launch in the United States (N. Virginia) Region

Na seção:

- **Name and tags**, usar *Control Plane* como **Name**.

Name and tags [Info](#)

Name

Control Plane

Add additional tags

- **Application and OS Images (Amazon Machine Image):**
 - Selecionar **Ubuntu**.
 - Em **Amazon Machine Image (AMI)**, selecionar **UbuntuServer 26.04 LTS (HVM), SSD Volume Type – Free tier eligible**
 - Em **Architecture**, selecionar **64-bit (x86)**

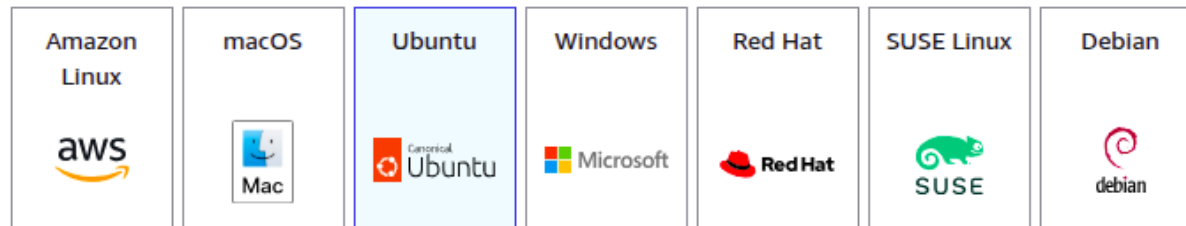
▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

 Search our full catalog including 1000s of application and OS images

Recents

Quick Start



[Browse more AMIs](#)

Including AMIs from
AWS, Marketplace and
the Community

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type

Free tier eligible ▼

ami-0b6d9d3d33ba97d99 (64-bit (x86)) / ami-0bc7f2dbdcc6b5303 (64-bit (Arm))

Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Ubuntu Server 26.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 26.04, amd64 resolute image

Architecture

64-bit (x86) ▼

AMI ID

ami-0b6d9d3d33ba97d99

Publish Date

2026-06-04

Username

ubuntu



Verified provider

- **Instance type**, selecionar **t3.small** - uma EC2 com 2 vCPU e 2 GB de memória (procurar por equivalente mais barato que seja **Free tier eligible**)

▼ **Instance type** [Info](#) | [Get advice](#)

Instance type

t3.small

Free tier eligible

Family: t3 2 vCPU 2 GiB Memory Current generation: true On-Demand RHEL base pricing: 0.0496 USD per Hour

On-Demand Windows base pricing: 0.0392 USD per Hour On-Demand SUSE base pricing: 0.0518 USD per Hour

On-Demand Linux base pricing: 0.0208 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0243 USD per Hour

☐ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

- **Key pair (login)**, clicar em **Create new key pair**. Na janela popup **Create key pair** que abrir:
 - Em **Key pair name**, informar *SmartFactory*
 - Em **Key pair type**, selecionar **RSA**
 - Em **Private key file format**, selecionar **.pem**
 - Clicar em **Create key pair** e salvar no diretório *KeyPair* do projeto (repositório)

The image shows a 'Create key pair' dialog box with a close button (X) in the top right corner. It contains three main sections: 'Key pair name' with a text input field containing 'SmartFactory' and a note about character limits; 'Key pair type' with two radio button options, 'RSA' (selected) and 'ED25519'; and 'Private key file format' with two radio button options, '.pem' (selected) and '.ppk'. A warning box at the bottom states: 'When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. Learn more'. At the bottom right are 'Cancel' and 'Create key pair' buttons.

Create key pair ✕

Key pair name
Key pairs allow you to connect to your instance securely.

SmartFactory

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn more](#)

[Cancel](#) **Create key pair**

- (de volta ao console **EC2**), em **Key pair name – required** informe o nome que você usou para criar a chave (*SmartFactory*).

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name – required

SmartFactory



[Create new key pair](#)

- **Network settings:**
 - Em **Firewall (security groups)**, selecione:
 - **Create security group.**
 - **Allow SSH traffic from Anywhere** (caixa ao lado)
 - **Allow HTTPS traffic from the internet**
 - **Allow HTTP traffic from the internet**

▼ **Network settings** [Info](#)

Network

[Info](#)

vpc-02d9756acd06c9137

Subnet

[Info](#)

No preference (Default subnet in any availability zone)

Auto-assign public IP

[Info](#)

Enable

Firewall (security groups)

[Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-1' with the following rules:

☒ Allow SSH traffic from

Helps you connect to your instance

Anywhere
0.0.0.0/0 ▼

☒ Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

☒ Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

⚠ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

×

- **Configure storage**, escolha **8 GiB**
 - Em **File systems**, selecione **None**

▼ **Configure storage** [Info](#)

Advanced

1x GiB ▼ Root volume, 3000 IOPS, Not encrypted

[Add new volume](#)

The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance

🕒 Click refresh to view backup information

The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

↻

File systems

☐ S3 Files - new

☐ EFS

☐ FSx

☒ None

- Nada precisa ser configurado em **Advanced details**.

- Clique em **Launch instance** no menu lateral direito.

▼ Summary

Number of instances | [Info](#)

1

Software Image (AMI)

Canonical, Ubuntu, 26.04, amd64...[read more](#)

ami-0b6d9d3d33ba97d99

Virtual server type (instance type)

t3.2xlarge

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 100 GiB

[Cancel](#)

Launch instance

 [Preview code](#)

Depois de criar a instância, inclua uma regra no **Security groups** para permitir acessos de entrada (**Inbound rules**):

- **TCP** porta 6443 (API do **K8s**)
- **UDP** porta 8472 (**VXLAN**)
- **TCP** porta 15672 (interface administrativa do **RabbitMQ**)
- **TCP** porta 8000 (interface da aplicação web)
- **TCP** porta 10250 (**Kubelet**)
- **TCP** porta 3000 (**Grafana**)
- **TCP** porta 9090 (**Prometheus**)

Inbound rules (10)

Manage tags

Edit inbound rules

< 1 >

Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	SSH	sgr-03d9aefa50b789847	IPv4	SSH	TCP	22	0.0.0.0/0	SSH
☐	HTTP	sgr-0f741378db7f2db3c	IPv4	HTTP	TCP	80	0.0.0.0/0	HTTP
☐	HTTPS	sgr-0ba0feba10e43554d	IPv4	HTTPS	TCP	443	0.0.0.0/0	HTTPS
☐	Grafana	sgr-078101799072f0b7b	IPv4	Custom TCP	TCP	3000	0.0.0.0/0	Grafana
☐	k8s	sgr-0644bd46b22f5b9d7	IPv4	Custom TCP	TCP	6443	0.0.0.0/0	API do k8s
☐	Web	sgr-05effcf80301733bd	IPv4	Custom TCP	TCP	8000	0.0.0.0/0	Web Application
☐	VXLAN	sgr-0bd839cfee7a06a9a	IPv4	Custom UDP	UDP	8472	0.0.0.0/0	VXLAN
☐	Prometheus	sgr-0f76cd1169c7034e2	IPv4	Custom TCP	TCP	9090	0.0.0.0/0	Prometheus
☐	Kubelet	sgr-0feb29f984e1f4d2	IPv4	Custom TCP	TCP	10250	0.0.0.0/0	Kubelet
☐	RabbitMQ	sgr-0d2d949e1eb57b582	IPv4	Custom TCP	TCP	15672	0.0.0.0/0	Interface administrativ...

Clique na instância criada.

▼ Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager [New](#)

Instances (3) [Info](#)

Last updated
less than a minute ago 

[Connect](#)

[Instance state ▼](#)

[Actions ▼](#)

[Launch instances ▼](#)

Saved filter sets

[Choose filter set ▼](#)

[Instance state = running](#) [X](#)

[Clear filters ▼](#)

[< 1 > ⚙](#)

☐	Name 🔗 ▼	Instance ID	Instance state ▼	Instance type ▼	Status check	Availability Zone ▼	Public IPv4 DNS ▼	Public IPv4 ... ▼	Elastic IP ▼	IP
☐	Kubernetes Examples 1	i-0c235f49bdc39a367	✓ Running 🔍 🔍	t3.small	✓ 3/3 checks passed	us-east-1c	ec2-3-235-11-145.com...	3.235.11.145	-	-

Volte para a instância do EC2, clicando em **Instances** e depois no ID da instância. Clique em **Connect**:



Na janela **Connect**, clique na aba “**In SSH client**” e siga as instruções.

```
Kubernetes/SmartFactory_Lab/KeyPair$ ssh -i "SmartFactory.pem" ubuntu@ec2-52-90-25-130.compute-1.amazonaws.com
The authenticity of host 'ec2-52-90-25-130.compute-1.amazonaws.com (52.90.25.130)' can't be established.
ED25519 key fingerprint is SHA256:uDx8V/bdRFDYrdV1W22og2l3UpS4XyupojxF2KWo+Y.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-52-90-25-130.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)
```

```
* Documentation:  https://docs.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro
```

System information as of Tue Jun 30 21:51:48 UTC 2026

System load:	0.17	Temperature:	-273.1 C
Usage of /:	2.1% of 95.85GB	Processes:	122
Memory usage:	3%	Users logged in:	0
Swap usage:	0%	IPv4 address for ens5:	172.31.46.4

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in `/usr/share/doc/*/copyright`.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Instalação das Aplicações Linux

Para desenvolver o projeto, você vai precisar de:

Aplicação	Pacote	host	Instalador
ifconfig	net-tools	Control Plane	apt
netstat			
docker	docker.io	Control Plane	apt
ensurepip	python3-venv	Control Plane	apt

Eventualmente você pode precisar também de:

Aplicação	Pacote	Instalador
wget	wget	apt

Instalação das Aplicações do Projeto

k3s

O *Control Plane* hospedará o **K8s Control Plane**:

```
$ curl -sfL https://get.k3s.io | sh -s - server
[INFO] Finding release for channel stable
[INFO] Using v1.36.2+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.36.2%2Bk3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.36.2%2Bk3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubectl symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s.service
[INFO] systemd: Enabling k3s unit
Created symlink '/etc/systemd/system/multi-user.target.wants/k3s.service' → '/etc/systemd/system/k3s.service'.
[INFO] systemd: Starting k3s
```

Comando:

- curl: acessa dados via URL
 - -s: silent
 - -f: não gera body em caso de falha
 - -L: segue o link em caso de mudança de local (erro 3XX)
 - https://get.k3s.io
- sh: interpretador de comandos shell
 - -s: ler o script da entrada padrão
 - -: a partir desse ponto, os parâmetros são para o script
 - server: instala o k3s no modo server

Verifique a instalação:

```
$ sudo k3s kubectl get nodes
NAME                STATUS    ROLES    AGE     VERSION
ip-172-31-0-13      Ready    control-plane  5m54s   v1.36.2+k3s1
```

Informações para os *workers*

No *Control Plane*, liste o *token* e guarde essa informação:

```
$ sudo cat /var/lib/rancher/k3s/server/node-token
K10554d2ac5b14d9d23ac4d2d0e0eaa9cf67e763d6a55d6db5cbb3bf526f254223c::server:db373843732b142a0de26a90d3467150
```

Obtenha também o endereço IP local. Você pode consultar o **Private IPv4 addresses** da **EC2** ou usar o comando `ifconfig`. Guarde também essa informação.

Private IPv4 addresses
 172.31.0.13

```
$ ifconfig
ens5: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 9001
    inet 172.31.0.13 netmask 255.255.240.0 broadcast 172.31.15.255
    inet6 fe80::c8:1bff:fe30:2ef1 prefixlen 64 scopeid 0x20<link>
    ether 02:c8:1b:30:2e:f1 txqueuelen 1000 (Ethernet)
    RX packets 116901 bytes 159763769 (159.7 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14549 bytes 1338321 (1.3 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Clonando

Clone o repositório INF1304:

```
$ git clone https://github.com/AlexandreMeslin/INF1304.git
Cloning into 'INF1304'...
remote: Enumerating objects: 8455, done.
remote: Counting objects: 100% (179/179), done.
remote: Compressing objects: 100% (122/122), done.
remote: Total 8455 (delta 82), reused 142 (delta 49), pack-reused 8276 (from 3)
Receiving objects: 100% (8455/8455), 113.84 MiB | 23.45 MiB/s, done.
Resolving deltas: 100% (1244/1244), done.
```

Remova todos os arquivos e diretórios menos a árvore abaixo de **SmartFactory_Lab**:

```
INF1304
├── Exemplos
│   └── Kubernetes
│       └── SmartFactory_Lab
│           ├── README.md
│           ├── k8s
│           │   ├── hpa.yaml
│           │   ├── ingress.yaml
│           │   ├── mysql.yaml
│           │   ├── namespace.yaml
│           │   ├── rabbitmq.yaml
│           │   └── web.yaml
│           ├── rebuild_k8s.sh
│           └── web
│               ├── Dockerfile
│               ├── factoryflow
│               │   ├── settings.py
│               │   ├── urls.py
│               │   └── views.py
│               ├── manage.py
│               └── requirements.txt
```

Descer para o diretório-raiz do projeto:

```
$ cd ~/INF1304/Exemplos/Kubernetes/SmartFactory_Lab/
```

Criar o ambiente base

Montando o núcleo do laboratório:

- Namespace
- MySQL
- RabbitMQ (com painel web)

Namespace

Aplicar o namespace:

```
$ sudo kubectl apply -f k8s/namespace.yaml
namespace/factoryflow created
```

Adicionar Worker 1

Crie uma **EC2** chamada *Worker-1*. Esta máquina irá hospedar o banco de dados **MySQL**. Use a mesma configuração da **EC2 Control Plane** e o mesmo **security group**.

Vamos instalar o **K3s** no modo **agent**. Para isso precisamos saber o **IP** da máquina que hospeda o *Control-Plane*, assim como o *token* de acesso. Para facilitar, vamos criar variáveis do **shell** com esses valores no *Worker-1* (copie os valores do *Control Plane*):

```
$ IP_PRIVADO_CP=172.31.0.13
$ TOKEN=K10554d2ac5b14d9d23ac4d2d0e0eaa9cf67e763d6a55d6db5cbb3bf526f254223c::server:db373843732b142a0de26a90d3467150
```

Teste a conectividade com o *Control Plane*:

```
$ curl -vk https://$IP_PRIVADO_CP:6443/version
Trying 172.31.0.13:6443...
ALPN: curl offers h2,http/1.1
TLSv1.3 (OUT), TLS handshake, Client hello (1):
SSL Trust: peer verification disabled
TLSv1.3 (IN), TLS handshake, Server hello (2):
TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
TLSv1.3 (IN), TLS handshake, Request CERT (13):
TLSv1.3 (IN), TLS handshake, Certificate (11):
TLSv1.3 (IN), TLS handshake, CERT verify (15):
TLSv1.3 (IN), TLS handshake, Finished (20):
TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
TLSv1.3 (OUT), TLS handshake, Certificate (11):
TLSv1.3 (OUT), TLS handshake, Finished (20):
SSL connection using TLSv1.3 / TLS_AES_128_GCM_SHA256 / X25519MLKEM768 / id-ecPublicKey
ALPN: server accepted h2
Server certificate:
subject: O=k3s; CN=k3s
start date: Jul 2 19:06:08 2026 GMT
expire date: Jul 2 19:06:08 2027 GMT
issuer: CN=k3s-server-ca@1783022768
Certificate level 0: Public key type EC/prime256v1 (256/128 Bits/secBits), signed using ecdsa-with-SHA256
Certificate level 1: Public key type EC/prime256v1 (256/128 Bits/secBits), signed using ecdsa-with-SHA256
SSL certificate verification failed, continuing anyway!
Established connection to 172.31.0.13 (172.31.0.13 port 6443) from 172.31.7.142 port 40344
using HTTP/2
[HTTP/2] [1] OPENED stream for https://172.31.0.13:6443/version
[HTTP/2] [1] [:method: GET]
[HTTP/2] [1] [:scheme: https]
[HTTP/2] [1] [:authority: 172.31.0.13:6443]
[HTTP/2] [1] [:path: /version]
[HTTP/2] [1] [user-agent: curl/8.18.0]
[HTTP/2] [1] [accept: /]
GET /version HTTP/2 Host: 172.31.0.13:6443 User-Agent: curl/8.18.0 Accept: /
Request completely sent off
TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
```

```
received GOAWAY, error=0, last_stream=1 < HTTP/2 401 < audit-id: ebf6b0de-0ea8-4df5-8d8d-d11ccd429882 < cache-control:
no-cache, private < content-type: application/json < content-length: 157 < date: Sat, 04 Jul 2026 21:16:24 GMT <
{ "kind": "Status", "apiVersion": "v1", "metadata": {}, "status": "Failure", "message": "Unauthorized", "reason":
"Unauthorized", "code": 401
shutting down connection #0 }
```

Instale o **k3s** no *worker-1* (note o sinal de menos no final do comando):

```
$ curl -sfL https://get.k3s.io | K3S_URL=https://$IP_PRIVADO_CP:6443 K3S_TOKEN=$TOKEN sh -
[INFO] Finding release for channel stable
[INFO] Using v1.36.2+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.36.2%2Bk3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.36.2%2Bk3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubect1 symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s-agent.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO] systemd: Enabling k3s-agent unit
Created symlink '/etc/systemd/system/multi-user.target.wants/k3s-agent.service' → '/etc/systemd/system/k3s-
agent.service'.
[INFO] systemd: Starting k3s-agent
```

Se você errou o comando, remova e tente novamente:

```
$ sudo /usr/local/bin/k3s-agent-uninstall.sh
```

Verifique se o processo não existe mais:

```
$ ps -ef | grep k3s
ubuntu      2125      1361    0 22:11 pts/0    00:00:00 grep --color=auto k3s
```

Verifique a instalação executando o seguinte comando no *Control Plane*:

```
$ sudo kubectl get nodes
NAME                STATUS    ROLES          AGE   VERSION
ip-172-31-0-13      Ready    control-plane  2d1h  v1.36.2+k3s1
ip-172-31-7-142     Ready    <none>         12m   v1.36.2+k3s1
```

MySQL

Vamos criar um *label* para o *Worker-1*. Primeiro, verifique o nome do *Worker-1*:

```
$ sudo kubectl get nodes -o wide
NAME                STATUS    ROLES          AGE   VERSION          INTERNAL-IP    EXTERNAL-IP    OS-IMAGE
KERNEL-VERSION      CONTAINER-RUNTIME
ip-172-31-0-13      Ready    control-plane  2d1h  v1.36.2+k3s1     172.31.0.13    <none>         Ubuntu 26.04 LTS  7.0.0-1006-aws (amd64) containerd://2.3.2-k3s2
ip-172-31-7-142     Ready    <none>         28m   v1.36.2+k3s1     172.31.7.142   <none>         Ubuntu 26.04 LTS  7.0.0-1006-aws (amd64) containerd://2.3.2-k3s2
```

Crie o label:

```
$ sudo kubectl label node ip-172-31-7-142 mysql=true
node/ip-172-31-7-142 labeled
```

Edite o arquivo `k8s/mysql.yaml` e inclua a cláusula `nodeSelector`, configurando-a para selecionar apenas os nós cujo rótulo `mysql` tenha o valor `"true"`.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
  namespace: factoryflow
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      nodeSelector:
        mysql: "true"
```

Aplicar:

```
$ sudo kubectl apply -f k8s/mysql.yaml
secret/mysql-secret created
deployment.apps/mysql-deployment created
service/mysql-service created
```

Verifique se o *pod* foi criado corretamente e em qual **EC2**:

```
$ sudo kubectl get pods -n factoryflow -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
READINESS GATES							
mysql-deployment-6c44ddf8fd-7hxm6	1/1	<u>Running</u>	0	30s	10.42.1.3	<u>ip-172-31-7-142</u>	<none>
<none>							

Para remover o *pod*:

```
$ sudo kubectl delete -f k8s/mysql.yaml
secret "mysql-secret" deleted from factoryflow namespace
deployment.apps "mysql-deployment" deleted from factoryflow namespace
service "mysql-service" deleted from factoryflow namespace
```

RabbitMQ

Crie mais uma **EC2**, agora chamada *Worker-2*, e faça todo o processo para o **RabbitMQ**.

Em *Worker-2*:

```
$ TOKEN=K1069c774e4ce3ddd9d25508bfe2397d3b2efa5812c9bab31782dd32a2ecc1dd6cb::server:c261d182c560d6e935ca1672dbe49a52
$ IP_PRIVADO_CP=172.31.27.229
$ curl -vk https://$IP_PRIVADO_CP:6443/version
$ curl -sfL https://get.k3s.io | K3S_URL=https://$IP_PRIVADO_CP:6443 K3S_TOKEN=$TOKEN sh -
```


Vamos criar um *label* para o *Worker-2*. Primeiro, verifique o nome do *Worker-2* no *Control Plane*:

```
$ sudo kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
KERNEL-VERSION		CONTAINER-RUNTIME					
ip-172-31-25-53	Ready	<none>	5h53m	v1.36.2+k3s1	172.31.25.53	<none>	Ubuntu 26.04 LTS
7.0.0-1006-aws (amd64)		containerd://2.3.2-k3s2					
ip-172-31-27-229	Ready	control-plane	6h7m	v1.36.2+k3s1	172.31.27.229	<none>	Ubuntu 26.04 LTS
7.0.0-1006-aws (amd64)		containerd://2.3.2-k3s2					
<u>ip-172-31-30-33</u>	Ready	<none>	5h35m	v1.36.2+k3s1	172.31.30.33	<none>	Ubuntu 26.04 LTS
7.0.0-1006-aws (amd64)		containerd://2.3.2-k3s2					

Crie o label:

```
$ sudo kubectl label node ip-172-31-30-33 rabbitmq=true
node/ip-172-31-7-142 labeled
```

Edite o arquivo `k8s/rabbitmq.yaml` e inclua a cláusula `nodeSelector`, configurando-a para selecionar apenas os nós cujo rótulo `rabbitmq` tenha o valor `"true"`.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rabbitmq-deployment
  namespace: factoryflow
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: rabbitmq
  template:
    metadata:
      labels:
        app: rabbitmq
    spec:
      nodeSelector:
        rabbitmq: "true"
```

Ao terminar, faça o *deployment* do **RabbitMQ**.

```
$ sudo kubectl apply -f k8s/rabbitmq.yaml
secret/rabbitmq-secret created
deployment.apps/rabbitmq-deployment created
service/rabbitmq-service created
```

Verifique se os *Pods* estão no ar:

```
$ sudo kubectl get pods -n factoryflow -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
mysql-deployment-5df99d9985-5fjvw	1/1	Running	0	8m16s	10.42.1.4	ip-172-31-7-142	<none>
<none>							
<u>rabbitmq-deployment-7b968d6dd8-chjdz</u>	1/1	Running	0	31s	10.42.2.3	<u>ip-172-31-12-96</u>	<none>
<none>							

Verifique também quais serviços estão no ar:

```
$ sudo kubectl get services -n factoryflow
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
mysql-service	ClusterIP	10.43.217.51	<none>	3306/TCP	5m29s
rabbitmq-service	ClusterIP	10.43.176.172	<none>	5672/TCP,15672/TCP	3m14s

Conectividade

Teste o **DNS** interno. No *Control Plane*, crie uma instância de uma máquina para verificar se o **DNS** interno está resolvendo todos os nomes e endereços.

```
$ sudo kubectl run debug --rm -it --image=busybox -n factoryflow -- sh
All commands and output from this session will be recorded in container logs, including credentials and sensitive
information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
/ #
```

Utilizando o pod, teste o DNS com os seguintes comandos:

```
/ # nslookup rabbitmq-service  
/ # nslookup mysql-service
```

Resultado esperado:

```
/ # nslookup rabbitmq-service  
Server:      10.43.0.10  
Address:     10.43.0.10:53  
  
Name:        rabbitmq-service.factoryflow.svc.cluster.local  
Address: 10.43.176.172  
  
** server can't find rabbitmq-service.svc.cluster.local: NXDOMAIN  
  
** server can't find rabbitmq-service.svc.cluster.local: NXDOMAIN  
  
** server can't find rabbitmq-service.cluster.local: NXDOMAIN  
  
** server can't find rabbitmq-service.cluster.local: NXDOMAIN  
  
** server can't find rabbitmq-service.ec2.internal: NXDOMAIN  
  
** server can't find rabbitmq-service.ec2.internal: NXDOMAIN  
  
/ # nslookup mysql-service  
Server:      10.43.0.10  
Address:     10.43.0.10:53  
  
** server can't find mysql-service.cluster.local: NXDOMAIN  
  
Name:        mysql-service.factoryflow.svc.cluster.local  
Address: 10.43.217.51  
  
** server can't find mysql-service.svc.cluster.local: NXDOMAIN
```

Para remover o *pod* (em caso de erro):

```
$ sudo kubectl delete pod debug -n factoryflow
pod "debug" deleted from factoryflow namespace
```

Observabilidade

Verificar o painel do **RabbitMQ**. Verifique que a porta **15672** do **RabbitMQ** está sendo exposta somente internamente ao cluster.

```
$ sudo netstat -tulpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      1/init
tcp        0      0 127.0.0.54:53          0.0.0.0:*               LISTEN      182/systemd-resolve
tcp        0      0 127.0.0.1:10249        0.0.0.0:*               LISTEN      696/k3s server
tcp        0      0 127.0.0.1:10248        0.0.0.0:*               LISTEN      696/k3s server
tcp        0      0 127.0.0.1:10259        0.0.0.0:*               LISTEN      696/k3s server
tcp        0      0 127.0.0.1:10258        0.0.0.0:*               LISTEN      696/k3s server
tcp        0      0 127.0.0.1:10257        0.0.0.0:*               LISTEN      696/k3s server
tcp        0      0 127.0.0.1:10256        0.0.0.0:*               LISTEN      696/k3s server
tcp        0      0 127.0.0.1:6444         0.0.0.0:*               LISTEN      696/k3s server
tcp        0      0 127.0.0.1:10010        0.0.0.0:*               LISTEN      974/containerd
tcp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN      182/systemd-resolve
tcp6       0      0 :::22                  :::*                   LISTEN      1/init
tcp6       0      0 :::10250                :::*                   LISTEN      696/k3s server
tcp6       0      0 :::6443                 :::*                   LISTEN      696/k3s server
udp        0      0 127.0.0.54:53          0.0.0.0:*               LISTEN      182/systemd-resolve
udp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN      182/systemd-resolve
udp        0      0 172.31.89.169:68       0.0.0.0:*               LISTEN      552/systemd-network
udp        0      0 0.0.0.0:8472           0.0.0.0:*               LISTEN      -
udp        0      0 127.0.0.1:323          0.0.0.0:*               LISTEN      720/chronyd
udp6       0      0 :::1:323               :::*                   LISTEN      720/chronyd
```

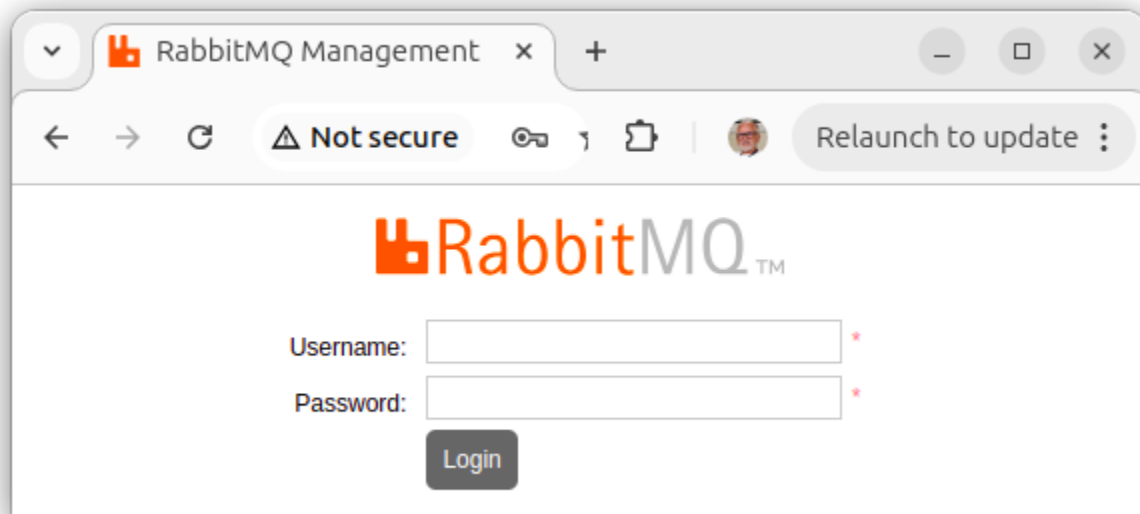
Vamos fazer *port forward* de dentro do cluster para fora da **EC2**:

```
$ sudo kubectl port-forward --address 0.0.0.0 -n factoryflow svc/rabbitmq-service 15672:15672
Forwarding from 0.0.0.0:15672 -> 15672
```

Comando:

- kubectl:
 - port-forward: criar túnel temporário
 - --address 0.0.0.0: endereço IP da(s) interface(s) físicas que estarão no túnel
 - -n factoryflow: namespace
 - svc/rabbitmq-service: conectar o serviço rabbitmq-service
 - 15672:15672: Porta do host : porta do serviço Kubernetes

Abra um navegador com o endereço público da **EC2 Control Plane (Public DNS)** e a porta 15672. Use as credenciais configuradas no arquivo `rabbitmq.yaml` (admin/admin123):



Examine o RabbitMQ:

The screenshot shows the RabbitMQ Management interface in a web browser. The browser's address bar shows the URL `ec2-2-235-11-145.compute-1.amazonaws.com:15672/#/`. The page title is "RabbitMQ Management". The interface includes a navigation bar with tabs: Overview, Connections, Channels, Exchanges, Queues and Streams, and Admin. The "Overview" tab is selected. The main content area shows the "Overview" section with a "Totals" subsection. It displays several statistics: "Queued messages" (last minute), "Currently idle", "Message rates" (last minute), "Currently idle", and "Global counts". Below these are buttons for "Connections: 0", "Channels: 0", "Exchanges: 7", "Queues: 0", and "Consumers: 0". The "Nodes" section is expanded, showing a table of nodes. The table has columns: Name, File descriptors, Socket descriptors, Erlang processes, Memory, Disk space, Uptime, Cores, Info, and Reset stats. There is one node listed: "rabbit@rabbitmq-deployment-7b968d6dd8-chjdz". The table also includes a "Churn statistics" section with links for "Ports and contexts", "Export definitions", and "Import definitions". At the bottom of the page, there is a footer with links: HTTP API, Documentation, Tutorials, New releases, Commercial edition, Commercial support, Discussions, Discord, Plugins, and GitHub.

RabbitMQ Management x +

Not secure ec2-2-235-11-145.compute-1.amazonaws.com:15672/#/ Relaunch to update

Refreshed 2026-07-04 20:12:08 Refresh every 5 seconds

Virtual host All

Cluster rabbit@rabbitmq-deployment-7b968d6dd8-chjdz User admin Log out

Overview Connections Channels Exchanges Queues and Streams Admin

Overview

Totals

Queued messages last minute ?

Currently idle

Message rates last minute ?

Currently idle

Global counts ?

Connections: 0 Channels: 0 Exchanges: 7 Queues: 0 Consumers: 0

Nodes

Name	File descriptors ?	Socket descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Cores	Info	Reset stats
rabbit@rabbitmq-deployment-7b968d6dd8-chjdz	44 1048576 available	0 943629 available	431 1048576 available	148 MiB 762 MiB high watermark	93 GiB 48 MiB low watermark	36m 7s	2	basic 2 rss	This node All nodes

Churn statistics

Ports and contexts

Export definitions

Import definitions

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

Veja detalhes sobre o uso da interface administrativa do **RabbitMQ** no documento “**Projeto SmartFactory Lab – RabbitMQ**”.

Aplicação Web

Crie uma **EC2** chamada *Worker-3* para fazer o *deploy* da aplicação web.

Em *Worker-3*:

```
$ TOKEN=K1069c774e4ce3ddd9d25508bfe2397d3b2efa5812c9bab31782dd32a2ecc1dd6cb::server:c261d182c560d6e935ca1672dbe49a52
$ IP_PRIVADO_CP=172.31.27.229
$ curl -vk https://$IP_PRIVADO_CP:6443/version
$ curl -sfL https://get.k3s.io | K3S_URL=https://$IP_PRIVADO_CP:6443 K3S_TOKEN=$TOKEN sh -
```

No Control Plane:

```
$ sudo kubectl get nodes
```

A aplicação web utiliza uma imagem **Docker** customizada. Podemos armazenar a imagem no **Docker Hub**, criar um **registry** no *Control Plane*, ou usar o **ECR da AWS** (difícil na academia).

Usando o **Docker Hub**:

Fazer login:

```
$ sudo docker login -u <seu usuário>
```

```
i Info → A Personal Access Token (PAT) can be used instead.
      To create a PAT, visit https://app.docker.com/settings
```

Password:

```
WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
```

Login Succeeded

Crie a imagem:

```
$ sudo docker build -t factoryflow-web:v1 ./web/
```

Verifique se a imagem foi gerada corretamente:

```
$ sudo docker images
```

					i	Info →	U	In Use
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA				
factoryflow-web:v1	bdc9fd6ebab5	599MB	152MB					
python:3.12-slim	423ed6ab25b1	179MB	45.4MB					

Suba a imagem:

```
$ sudo docker tag factoryflow-web:v1 meslin/factoryflow-web:v1
$ sudo docker push meslin/factoryflow-web:v1
The push refers to repository [docker.io/meslin/factoryflow-web]
3d395d07ad5f: Pushed
df79d931cd67: Mounted from library/python
b32430367bf0: Mounted from library/python
a434c51d3305: Pushed
309062f62937: Pushed
a5a4c082e13f: Pushed
e95a6c7ea7d4: Mounted from library/python
aff2d9f8dc87: Mounted from library/python
6a23791bc66b: Pushed
v1: digest: sha256:bdc9fd6ebab5541f95adac9fa244d582165445baf3bf61688245f24dcbe14895 size: 2275
```

Vamos criar um *label* para o *Worker-3*. Primeiro, verifique o nome do *Worker-3*:

```
$ sudo kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
KERNEL-VERSION		CONTAINER-RUNTIME					
ip-172-31-25-53	Ready	<none>	6h5m	v1.36.2+k3s1	172.31.25.53	<none>	Ubuntu 26.04 LTS
7.0.0-1006-aws (amd64)		containerd://2.3.2-k3s2					
ip-172-31-27-229	Ready	control-plane	6h19m	v1.36.2+k3s1	172.31.27.229	<none>	Ubuntu 26.04 LTS
7.0.0-1006-aws (amd64)		containerd://2.3.2-k3s2					
<u>ip-172-31-28-249</u>	Ready	<none>	28m	v1.36.2+k3s1	172.31.28.249	<none>	Ubuntu 26.04 LTS
7.0.0-1006-aws (amd64)		containerd://2.3.2-k3s2					
ip-172-31-30-33	Ready	<none>	5h47m	v1.36.2+k3s1	172.31.30.33	<none>	Ubuntu 26.04 LTS
7.0.0-1006-aws (amd64)		containerd://2.3.2-k3s2					

Crie o label:

```
$ sudo kubectl label node ip-172-31-28-249 web=true
node/ip-172-31-7-142 labeled
```

Alterar o *deployment*. Edite o arquivo `k8s/web.yaml` e inclua a cláusula `nodeSelector`, configurando-a para selecionar apenas os nós cujo rótulo `mysql` tenha o valor `"true"`, troque o nome da imagem para `meslin/factoryflow:v1` (ou o nome que você estiver utilizando no seu **Docker Hub**) e a política de *download* da imagem para `Always` (bom para depuração, depois pode trocar para `IfNotPresent`). Informe também o novo seletor usando a cláusula `nodeSelector` com `web` com a *string* `true`.

```
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: web # rótulo para identificar os pods deste deployment
  template:
    metadata:
      labels:
        app: web # rótulo para identificar os pods deste deployment
    spec:
      nodeSelector:
        web: "true"
      containers:
        - name: web # nome do container
          image: meslin/factoryflow-web:v1
          imagePullPolicy: Always
```

Faça o deploy:

```
$ sudo kubectl apply -f k8s/web.yaml
deployment.apps/web-deployment created
service/web-service created
```

Sempre que modificar a imagem, sem modificar o arquivo YAML, realizar o redeploy:

```
$ sudo kubectl rollout restart deployment web-deployment -n factoryflow  
deployment.apps/web-deployment restarted
```

Verifique o deploy:

```
$ sudo kubectl get pods -n factoryflow
```

NAME	READY	STATUS	RESTARTS	AGE
mysql-deployment-5df99d9985-5fjvw	1/1	Running	2 (45m ago)	25h
rabbitmq-deployment-7b968d6dd8-chjdz	1/1	Running	2 (45m ago)	24h
web-deployment-5d475c8b5b-7sd8p	1/1	Running	0	112s
web-deployment-5d475c8b5b-m8jwn	1/1	Running	0	112s

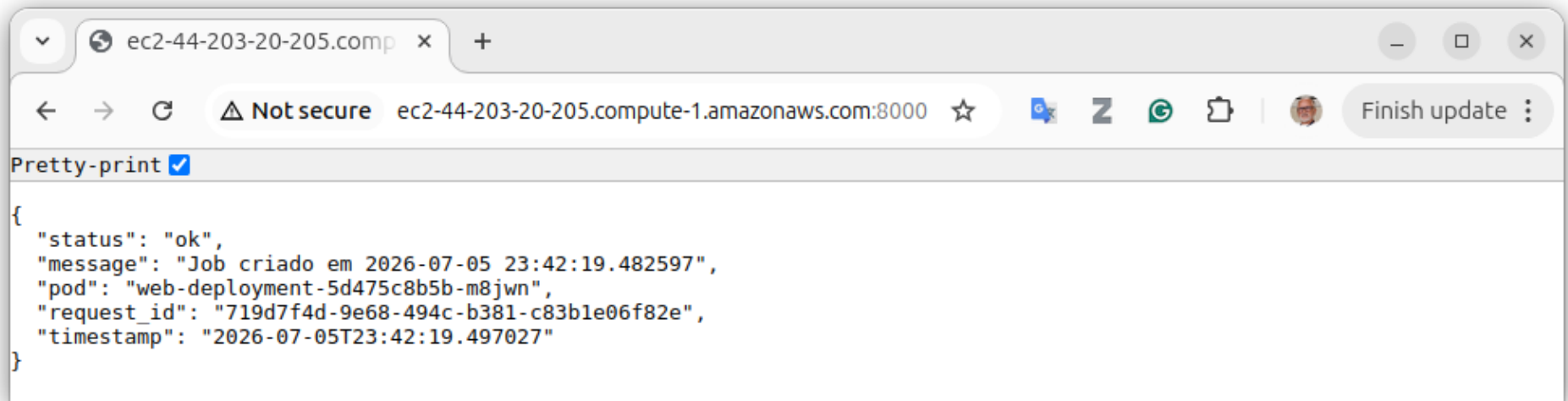
É necessário realizar *forward* da porta para que o serviço web seja disponibilizado externamente:

```
$ sudo kubectl port-forward --address 0.0.0.0 -n factoryflow svc/web-service 8000:80  
Forwarding from 0.0.0.0:8000 -> 8000
```

kubectl: cliente da API do K8s

- port-forward: cria um túnel
- --address 0.0.0.0: escuta em todas as interfaces da EC2
- -n factoryflow: namespace
- svc/web-service: serviço destino do túnel
- 8000:80: porta local:porta do serviço

Acesse o serviço web através de um *browser* no endereço público do **EC2 Control Plane** na porta **8000**:



Vamos usar o **ingress** para não precisar realizar *port forward*. Edite o arquivo `k8s/ingress.yaml` para usar o **traefik**:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: factoryflow-ingress
  namespace: factoryflow
spec:
  ingressClassName: traefik
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```



Traefik (pronounced traffic) is a modern HTTP reverse proxy and load balancer that makes deploying microservices easy. Traefik integrates with your existing infrastructure components (Docker, Swarm mode, Kubernetes, Consul, Etcd, Rancher v2, Amazon ECS, ...) and configures itself automatically and dynamically. Pointing Traefik at your orchestrator should be the only configuration step you need.

- <https://traefik.io/>
- <https://github.com/traefik/traefik>

Aplique o *deploy* do **ingress**:

```
$ sudo kubectl apply -f k8s/ingress.yaml
ingress.networking.k8s.io/factoryflow-ingress configured
```

Verifique a porta:

```
$ sudo kubectl get ingress -n factoryflow
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
factoryflow-ingress	traefik	*	172.31.0.13,172.31.12.96,172.31.4.241,172.31.7.142	80	9m17s

E verifique se o **traefik** está trabalhando como balanceador de carga:

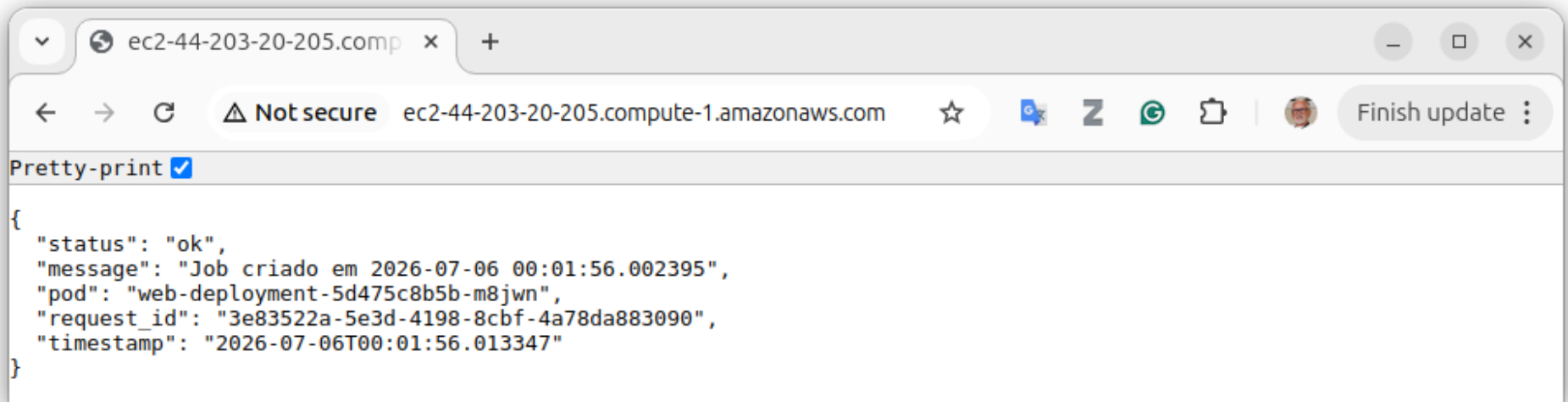
```
$ sudo kubectl get svc -n kube-system
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kube-dns	ClusterIP	10.43.0.10	<none>	
53/UDP,53/TCP,9153/TCP 3d3h				
metrics-server	ClusterIP	10.43.87.180	<none>	443/TCP
3d3h				
traefik	LoadBalancer	10.43.112.173	172.31.0.13,172.31.12.96,172.31.4.241,172.31.7.142	
80:31396/TCP,443:30456/TCP 3d3h				

Teste localmente:

```
$ curl http://localhost
{"status": "ok", "message": "Job criado em 2026-07-06 00:02:42.582853", "pod": "web-deployment-5d475c8b5b-m8jwn", "request_id": "06073dfd-b89d-4162-83ec-37ae0fce209d", "timestamp": "2026-07-06T00:02:42.590518"}
```

Teste no navegador, agora na porta 80:



Escalabilidade, Elasticidade, Load Balancing, Resiliência e Failover

Demonstrar:

- Controle da quantidade de pods
- Elasticidade de pods
- Balanceamento automático
- Recuperação após falha

Escalar o deployment

Verifique a quantidade de pods em execução:

```
$ sudo kubectl get pods -n factoryflow -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
NODE READINESS GATES							
mysql-deployment-5df99d9985-5fjvw	1/1	Running	4 (10m ago)	44h	10.42.1.12	ip-172-31-7-142	<none>
<none>							
rabbitmq-deployment-7b968d6dd8-chjdz	1/1	Running	4 (10m ago)	44h	10.42.2.15	ip-172-31-12-96	<none>
<none>							
web-deployment-5d475c8b5b-7sd8p	1/1	Running	2 (10m ago)	19h	10.42.3.9	ip-172-31-4-241	<none>
<none>							
web-deployment-5d475c8b5b-m8jwn	1/1	Running	3 (10m ago)	19h	10.42.3.10	ip-172-31-4-241	<none>
<none>							

Aumente para 3 a quantidade de pods web:

```
$ sudo kubectl scale deployment web-deployment --replicas=3 -n factoryflow
deployment.apps/web-deployment scaled
```

Verifique novamente a quantidade de pods:

```
$ sudo kubectl get pods -n factoryflow -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
mysql-deployment-5df99d9985-5fjvw	1/1	Running	4 (10m ago)	44h	10.42.1.12	ip-172-31-7-142	<none>
<none>							
rabbitmq-deployment-7b968d6dd8-chjdz	1/1	Running	4 (10m ago)	44h	10.42.2.15	ip-172-31-12-96	<none>
<none>							
web-deployment-5d475c8b5b-7sd8p	1/1	Running	2 (10m ago)	19h	10.42.3.9	ip-172-31-4-241	<none>
<none>							
web-deployment-5d475c8b5b-m8jwn	1/1	Running	3 (10m ago)	19h	10.42.3.10	ip-172-31-4-241	<none>
<none>							
web-deployment-5d475c8b5b-tlc6v	0/1	Running	0	4s	10.42.3.11	ip-172-31-4-241	<none>
<none>							

Balanceamento de carga

Entenda o *endpoint* em `/api/v1/health/`.

Realize vários acessos em sequência a partir da internet (sua máquina local):

```
$ PUBLIC_DNS_CP=ec2-54-242-105-135.compute-1.amazonaws.com
$ for i in {1..10}; do curl http://$PUBLIC_DNS_CP/api/v1/health/; echo; done
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T18:50:44.605847", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-m8jwn", "timestamp": "2026-07-06T18:50:44.912722", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T18:50:45.212888", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-m8jwn", "timestamp": "2026-07-06T18:50:45.531456", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T18:50:45.851031", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T18:50:46.158681", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T18:50:46.460686", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T18:50:46.780663", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-m8jwn", "timestamp": "2026-07-06T18:50:47.089305", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T18:50:47.390525", "version": "2"}
```

Teste de *failover*

Verifique quais pods estão no ar:

```
$ sudo kubectl get pods -n factoryflow
```

NAME	READY	STATUS	RESTARTS	AGE
mysql-deployment-5df99d9985-5fjvw	1/1	Running	4 (21m ago)	44h
rabbitmq-deployment-7b968d6dd8-chjdz	1/1	Running	4 (21m ago)	44h
web-deployment-5d475c8b5b-7sd8p	1/1	Running	2 (21m ago)	19h
web-deployment-5d475c8b5b-m8jwn	1/1	Running	3 (21m ago)	19h
web-deployment-5d475c8b5b-tlc6v	1/1	Running	0	10m

Remova um dos pods web:

```
$ sudo kubectl delete pod -n factoryflow web-deployment-5d475c8b5b-m8jwn
pod "web-deployment-5d475c8b5b-m8jwn" deleted from factoryflow namespace
```

Verifique a situação dos pods. Note que foi criado um novo pod para substituir o pod que foi removido.

```
$ sudo kubectl get pods -n factoryflow -w
```

NAME	READY	STATUS	RESTARTS	AGE
mysql-deployment-5df99d9985-5fjvw	1/1	Running	4 (22m ago)	44h
rabbitmq-deployment-7b968d6dd8-chjdz	1/1	Running	4 (22m ago)	44h
web-deployment-5d475c8b5b-7sd8p	1/1	Running	2 (22m ago)	19h
web-deployment-5d475c8b5b-ct7c9	1/1	Running	0	21s
web-deployment-5d475c8b5b-tlc6v	1/1	Running	0	11m

Resiliência

Acesse o serviço pela internet usando um terminal na sua máquina local.

```
$ PUBLIC_DNS_CP=ec2-54-242-105-135.compute-1.amazonaws.com
$ while true; do curl http://$PUBLIC_DNS_CP/api/v1/health/; sleep 1; echo; done
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:11.282908", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:12.635657", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:14.368958", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:15.719940", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:17.055528", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:18.400845", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:19.752477", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:21.693976", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:23.134712", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:24.474820", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:25.817370", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:27.152618", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:28.484575", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:29.991410", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:31.404550", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:32.761237", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:34.107880", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:35.513723", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:36.861819", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:38.185388", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:39.505312", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:40.839599", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:42.189975", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:43.527804", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:45.174585", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:46.506969", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:47.850404", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:49.204276", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:50.537328", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:51.875059", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:53.213824", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:54.538961", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:55.882688", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:57.206594", "version": "2"}
```

```
{"status": "ok", "pod": "web-deployment-5d475c8b5b-k66kk", "timestamp": "2026-07-06T19:07:58.537314", "version": "2"}  
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:59.875000", "version": "2"}
```

Em paralelo, no *Control Panel*, remova um dos *pods* web:

```
$ sudo kubectl get pods -n factoryflow  
NAME                                READY   STATUS    RESTARTS   AGE  
mysql-deployment-5df99d9985-5fjvw   1/1     Running   4 (31m ago) 44h  
rabbitmq-deployment-7b968d6dd8-chjdz 1/1     Running   4 (31m ago) 44h  
web-deployment-5d475c8b5b-7sd8p     1/1     Running   2 (31m ago) 19h  
web-deployment-5d475c8b5b-ct7c9     1/1     Running   0           9m11s  
web-deployment-5d475c8b5b-tlc6v     1/1     Running   0           20m  
$ sudo kubectl delete pod -n factoryflow web-deployment-5d475c8b5b-7sd8p  
pod "web-deployment-5d475c8b5b-7sd8p" deleted from factoryflow namespace  
$ sudo kubectl get pods -n factoryflow  
NAME                                READY   STATUS    RESTARTS   AGE  
mysql-deployment-5df99d9985-5fjvw   1/1     Running   4 (32m ago) 44h  
rabbitmq-deployment-7b968d6dd8-chjdz 1/1     Running   4 (32m ago) 44h  
web-deployment-5d475c8b5b-ct7c9     1/1     Running   0           10m  
web-deployment-5d475c8b5b-k66kk     1/1     Running   0           41s  
web-deployment-5d475c8b5b-tlc6v     1/1     Running   0           21m
```

Veja no terminal da máquina local:

```
$ PUBLIC_DNS_CP=ec2-54-242-105-135.compute-1.amazonaws.com
$ while true; do curl http://$PUBLIC_DNS_CP/api/v1/health/; sleep 1; echo; done
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:39.505312", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:40.839599", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:42.189975", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:43.527804", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:45.174585", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-7sd8p", "timestamp": "2026-07-06T19:07:46.506969", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:47.850404", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:49.204276", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:50.537328", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:51.875059", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:53.213824", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:54.538961", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:55.882688", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:07:57.206594", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-k66kk", "timestamp": "2026-07-06T19:07:58.537314", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:07:59.875000", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-k66kk", "timestamp": "2026-07-06T19:08:01.592466", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:08:02.938769", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-ct7c9", "timestamp": "2026-07-06T19:08:04.267514", "version": "2"}
{"status": "ok", "pod": "web-deployment-5d475c8b5b-tlc6v", "timestamp": "2026-07-06T19:08:05.596442", "version": "2"}
```

Continuous Deployment

Demonstração de como fazer *upgrade* sem *downtime*.

Altere a versão do código na função `health` do arquivo `web/factoryflow/views.py`.

```
def health(request):  
    '''  
    Endpoint de health check para monitoramento do serviço.  
    Retorna um JSON com status "ok", nome do pod e timestamp atual.  
    '''  
    json_response = {  
        "status": "ok",  
        "pod": os.uname().nodename,  
        "timestamp": datetime.now().isoformat(),  
        'version': '2',  
    }  
    return JsonResponse(json_response)
```

Faça o rebuild da imagem e atualize o Docker Hub:

```
$ sudo docker build -t factoryflow-web:v2 ./web  
$ sudo docker tag factoryflow-web:v2 meslin/factoryflow-web:v2  
$ sudo docker push meslin/factoryflow-web:v2
```

Atualizar *deployment* editando o arquivo `k8s/web.yaml`. Modifique a versão da imagem para 2.

Aplicar a atualização:

```
$ sudo kubectl apply -f web.yaml -n factoryflow
```

Observar:

```
$ PUBLIC_DNS_CP=ec2-54-242-105-135.compute-1.amazonaws.com
$ while true; do curl http://$PUBLIC_DNS_CP/api/v1/health/; sleep 1; echo; done
{"status": "ok", "pod": "web-deployment-78f8997897-ngnf2", "timestamp": "2026-07-06T19:39:03.519282", "version": "2"}
{"status": "ok", "pod": "web-deployment-78f8997897-ngnf2", "timestamp": "2026-07-06T19:39:04.853892", "version": "2"}
{"status": "ok", "pod": "web-deployment-78f8997897-ngnf2", "timestamp": "2026-07-06T19:39:06.162716", "version": "2"}
{"status": "ok", "pod": "web-deployment-78f8997897-ngnf2", "timestamp": "2026-07-06T19:39:07.472715", "version": "2"}
{"status": "ok", "pod": "web-deployment-5dcc559bc9-8jsbr", "timestamp": "2026-07-06T19:39:40.091812", "version": "3"}
{"status": "ok", "pod": "web-deployment-5dcc559bc9-cfjh6", "timestamp": "2026-07-06T19:39:41.413777", "version": "3"}
{"status": "ok", "pod": "web-deployment-5dcc559bc9-8jsbr", "timestamp": "2026-07-06T19:39:43.024239", "version": "3"}
{"status": "ok", "pod": "web-deployment-5dcc559bc9-cfjh6", "timestamp": "2026-07-06T19:39:44.324249", "version": "3"}
{"status": "ok", "pod": "web-deployment-5dcc559bc9-8jsbr", "timestamp": "2026-07-06T19:39:45.637052", "version": "3"}
{"status": "ok", "pod": "web-deployment-5dcc559bc9-cfjh6", "timestamp": "2026-07-06T19:39:46.952764", "version": "3"}
```

Elasticidade (HPA - Horizontal Pod Autoscaler)

Antes de começar, verifique se a porta TCP 10250 está aberta nas EC2s. Confira os Security Groups.

Confirme que o **Metric Server** está no ar:


```
$ sudo kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-5f5694d56b-g7t7c	1/1	Running	7 (128m ago)	4d
helm-install-traefik-crd-hprrz	0/1	Completed	0	4d
helm-install-traefik-hsw95	0/1	Completed	2 (4d ago)	4d
local-path-provisioner-58d557dc48-xcv7v	1/1	Running	7 (128m ago)	4d
metrics-server-7c86f97b8d-wp884	1/1	Running	7 (128m ago)	4d
svclb-traefik-aa9d9c88-798ww	2/2	Running	14 (128m ago)	4d
svclb-traefik-aa9d9c88-8qjvn	2/2	Running	8 (127m ago)	46h
svclb-traefik-aa9d9c88-lhttq	2/2	Running	4 (127m ago)	21h
svclb-traefik-aa9d9c88-nw5m7	2/2	Running	8 (127m ago)	47h
traefik-6cd8c7cd89-4d6nb	1/1	Running	7 (128m ago)	4d

Aplique o HPA:

```
$ sudo kubectl apply -f k8s/hpa.yaml
```

```
horizontalpodautoscaler.autoscaling/web-hpa created
```

Vamos precisar de 2 terminais na **EC2** do *Control Plane*. No primeiro terminal, monitore a quantidade de *pods*:

```
$ watch -n1 "sudo kubectl get hpa,pods -n factoryflow"
```

No segundo terminal, instancie uma imagem para gerar tráfego:

```
$ sudo kubectl run load-generator --rm -it --image=busybox -n factoryflow -- sh
```

All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.

If you don't see a command prompt, try pressing enter.

```
/ #
```

kubectl: cliente k8s

- run load-generator: cria um novo pod chamado load-generator
- --rm remove o pod quando ele termina

- -it: abre a entrada padrão e a associa a um terminal tty
- --image=busybox: usa a imagem busybox
- -n factoryflow: namespace
- --: separador de comandos do kubectl e do container
- sh: o que será executado no container

No container, gere tráfego com o comando:

```
while true
do
    wget -q -O- http://web-service/api/v1/health/
done
```

wget:cliente http

- -q: modo silencioso
- -O-: especifica que o arquivo de saída será stdout (-)
- http://web-service/api/v1/health/: endpoint

Veja a quantidade de pods web aumentar no primeiro terminal.

\$ watch -n1 "sudo kubectl get hpa,pods -n factoryflow"						
NAME	REFERENCE			TARGETS	MINPODS	MAXPODS REPLICAS
AGE						
horizontalpodautoscaler.autoscaling/web-hpa	Deployment/web-deployment			cpu: 79%/50%	2	6 6
13m						
NAME	READY	STATUS	RESTARTS	AGE		
pod/load-generator	1/1	Running	0	94s		
pod/mysql-deployment-df7f867fb-8h2l6	1/1	Running	0	85m		
pod/rabbitmq-deployment-74f7988997-r2g6k	1/1	Running	0	85m		
pod/web-deployment-78f8997897-f8w2s	1/1	Running	0	42m		
pod/web-deployment-78f8997897-h4dqt	1/1	Running	0	43s		
pod/web-deployment-78f8997897-h7cmw	1/1	Running	0	42m		
pod/web-deployment-78f8997897-kmzjh	1/1	Running	0	29s		
pod/web-deployment-78f8997897-nkjdn	1/1	Running	0	43s		
pod/web-deployment-78f8997897-nxkrm	1/1	Running	0	29s		

Interrompa o loop com **Ctrl+C** e veja a quantidade de *Pods* diminuir.

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
AGE					
horizontalpodautoscaler.autoscaling/web-hpa	Deployment/web-deployment	cpu: 2%/50%	2	6	2
19m					

NAME	READY	STATUS	RESTARTS	AGE
pod/load-generator	1/1	Running	0	3m8s
pod/mysql-deployment-df7f867fb-8h2l6	1/1	Running	0	91m
pod/rabbitmq-deployment-74f7988997-r2g6k	1/1	Running	0	91m
pod/web-deployment-78f8997897-f8w2s	1/1	Running	0	48m
pod/web-deployment-78f8997897-h7cmw	1/1	Running	0	48m

Observabilidade

Instalação do Prometheus e do Grafana

Prometheus is a free software application for event monitoring and alerting. It records metrics in a time series database built using an HTTP pull model, supporting high dimensionality through key-value label pairs, flexible queries, and real-time alerting. The project is written in Go and licensed under the Apache 2.0 License, with source code available on GitHub.

([https://en.wikipedia.org/wiki/Prometheus_\(software\)](https://en.wikipedia.org/wiki/Prometheus_(software)))

Grafana is an open-source analytics and visualization web application. It connects to time series databases and other data sources, allowing users to build dashboards that display metrics, logs, and traces. Grafana supports data sources including Prometheus, AWS CloudWatch, Graphite, InfluxDB, Elasticsearch, PostgreSQL, and MySQL. (<https://en.wikipedia.org/wiki/Grafana>)

Helm is a package manager for Kubernetes. It uses 'charts' as its package format, which is based on YAML. It provides support for defining templates using the Go template syntax to generate Kubernetes resource manifests dynamically.

([https://en.wikipedia.org/wiki/Helm_\(package_manager\)](https://en.wikipedia.org/wiki/Helm_(package_manager)))

Instale o **helm** (<https://github.com/pkhamre/k3s-monitoring-stack>):

```
$ curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
Downloading https://get.helm.sh/helm-v3.21.3-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
```

Verifique se a instalação completou com sucesso:

```
$ helm version
version.BuildInfo{Version:"v3.21.3", GitCommit:"1ad6e68924fdf6fb0c7dcef8e9e1dfc0f36eae6", GitTreeState:"clean",
GoVersion:"go1.26.5"}
```

Mude o seu usuário para **root**:

```
$ sudo su -
```

Instale o **Prometheus** (<https://prometheus.io/docs/introduction/overview/>):

Adicione o repositório do **Prometheus** no **helm**:

```
# helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories

# helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. *Happy Helming!*
```

Crie o *namespace monitoring*:

```
# kubectl create namespace monitoring
namespace/monitoring created
```

Instale o *chart* kube-prometheus-stack pelo helm:

```
# helm --kubeconfig /etc/rancher/k3s/k3s.yaml install monitoring prometheus-community/kube-prometheus-stack --namespace
monitoring
NAME: monitoring
LAST DEPLOYED: Mon Jul  6 22:11:51 2026
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoring get pods -l "release=monitoring"

Get Grafana 'admin' user password by running:

  kubectl --namespace monitoring get secrets monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo

Access Grafana local instance:

  export POD_NAME=$(kubectl --namespace monitoring get pod -l
"app.kubernetes.io/name=grafana,app.kubernetes.io/instance=monitoring" -oname)
  kubectl --namespace monitoring port-forward $POD_NAME 3000

Get your grafana admin user password by running:

  kubectl get secret --namespace monitoring -l app.kubernetes.io/component=admin-secret -o
jsonpath="{.items[0].data.admin-password}" | base64 --decode ; echo

Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager
and Prometheus instances using the Operator.
```

Faça logout do usuário root e volte para o seu usuário. Verifique se o **Prometheus** e o **Grafana** estão rodando:

```
$ sudo kubectl get pods -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	0	5m45s
monitoring-grafana-7f96679fb5-wd94s	3/3	Running	0	5m50s
monitoring-kube-prometheus-operator-75696c5967-hmd8c	1/1	Running	0	5m50s
monitoring-kube-state-metrics-559db896bf-pqt8h	1/1	Running	0	5m50s
monitoring-prometheus-node-exporter-9fqqt	1/1	Running	0	5m51s
monitoring-prometheus-node-exporter-ddph6	1/1	Running	0	5m51s
monitoring-prometheus-node-exporter-h77ht	1/1	Running	0	5m51s
monitoring-prometheus-node-exporter-r7c4k	1/1	Running	0	5m51s
prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	0	5m44s

Verifique também os serviços:

```
$ sudo kubectl get svc -n monitoring
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
alertmanager-operated	ClusterIP	None	<none>	9093/TCP, 9094/TCP, 9094/UDP	6m19s
monitoring-grafana	ClusterIP	10.43.45.192	<none>	80/TCP	6m26s
monitoring-kube-prometheus-alertmanager	ClusterIP	10.43.40.32	<none>	9093/TCP, 8080/TCP	6m26s
monitoring-kube-prometheus-operator	ClusterIP	10.43.48.146	<none>	443/TCP	6m26s
monitoring-kube-prometheus-prometheus	ClusterIP	10.43.38.168	<none>	9090/TCP, 8080/TCP	6m26s
monitoring-kube-state-metrics	ClusterIP	10.43.89.128	<none>	8080/TCP	6m26s
monitoring-prometheus-node-exporter	ClusterIP	10.43.92.171	<none>	9100/TCP	6m26s
prometheus-operated	ClusterIP	None	<none>	9090/TCP	6m18s

Tenha certeza de abrir as portas 3000 (**Grafana**) e 9090 (**Prometheus**) na **EC2** que hospeda o **Prometheus** e o **Grafana**.

Para recuperar a senha do **Grafana**:

```
$ sudo kubectl --namespace monitoring get secrets monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d ;  
echo  
Nblth2hcFObrWsb532VzoOcfR4LSq6Pzh6M7kL9p
```

Faça *port forward* para o **Grafana**:

```
$ sudo kubectl port-forward --address 0.0.0.0 svc/monitoring-grafana 3000:80 -n monitoring
Forwarding from 0.0.0.0:3000 -> 3000
```

Para acessar o **Grafana**, abra um navegador, use o endereço público da **EC2** que hospeda o **Control Plane** e a porta 3000. Se for (e será) necessário entrar com credenciais, use **admin** como usuário e obtenha a senha como mostrado durante a instalação do **Grafana**.

Faça *port forward* também para o **Prometheus**:

```
$ sudo kubectl port-forward --address 0.0.0.0 svc/monitoring-kube-prometheus-prometheus 9090:9090 -n monitoring
Forwarding from 0.0.0.0:9090 -> 9090
```

Para monitorar o RabbitMQ. Crie o arquivo `k8s/rabbitmq-servicemonitor.yaml`:


```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: rabbitmq-monitor
  namespace: monitoring
  labels:
    release: monitoring
spec:
  namespaceSelector:
    matchNames:
      - factoryflow

  selector:
    matchLabels:
      app: rabbitmq

  endpoints:
    - port: prometheus
      path: /metrics
      interval: 15s
```

Aplique o serviço:

```
$ sudo kubectl apply -f k8s/rabbitmq-servicemonitor.yaml
servicemonitor.monitoring.coreos.com/rabbitmq-monitor created
```

Habilitar o plugin Prometheus no RabbitMQ (não tenho certeza se é realmente necessário):

```
$ rabbitmq-plugins enable rabbitmq_prometheus
```

Verifique se o **Prometheus** encontrou o serviço de métricas do **RabbitMQ**. Vá em `Status > Target health` e procure pelo **RabbitMQ**:

serviceMonitor/monitoring/rabbitmq-monitor/0				1 / 1 up 	
Endpoint	Labels			Last scrape	State
http://10.42.2.31:15692/metrics	endpoint="prometheus"	instance="10.42.2.31:15692"	job="rabbitmq-service"	namespace="factoryflow"	
	pod="rabbitmq-deployment-7b968d6dd8-np9rj"	service="rabbitmq-service"		8.899s ago	12ms
					UP

Na raiz do **Prometheus**, clique em **Graph** e escolha uma métrica, por exemplo `rabbitmq_connections`, `rabbitmq_queue_messages` ou `rabbitmq_queue_messages_ready`.

No Grafana, clique em **Explore** no menu lateral esquerdo e depois selecione alguma métrica do RabbitMQ, por exemplo, `rabbitmq_connections` e depois clique em **Run query**.

Se tudo estiver correto, vamos criar algumas conexões.

Em um terminal, faça port-forward do RabbitMQ para que ele possa ser acessado por uma aplicação fora do cluster k8s:

```
$ sudo kubectl port-forward -n factoryflow svc/rabbitmq-service 5672:5672
Forwarding from 127.0.0.1:5672 -> 5672
Forwarding from [::1]:5672 -> 5672
```

Em um segundo terminal, crie e ative o ambiente virtual, instale os requisitos e execute o consumidor:

```
$ python3 -m venv venv
$ source venv/bin/activate
$ pip install -r requirements.txt
$ python3 consumer.py
```

Agora, em um terceiro terminal, crie e ative o ambiente virtual, instale os requisitos e execute o produtor:

```
$ python3 -m venv venv
$ source venv/bin/activate
$ pip install -r requirements.txt
$ python3 producer.py
```

Veja o resultado no Grafana:

